

RVME: An Efficient Matrix Engine Design Based on Matrix Extension of RISC-V

Wanqi Chen*, Weidong Yang*, Yiming Guo*, Jing Qiu[†], Renpei Wang*,
Jianfei Jiang*, Naifeng Jing* and Qin Wang*

*Shanghai Jiao Tong University, Shanghai, China, [†]Alibaba DAMO Academy, Hangzhou, China
{chenwanqi99, ywdyang, gym_ing, wangrenpei, jiangjianfei, sjtuj, qinqinwang}@sjtu.edu.cn
duojing.qj@alibaba-inc.com

Abstract—The rapid advancement of Deep Neural Networks (DNNs) continues to challenge traditional computing architectures, prompting the development of various hardware accelerators. However, insufficient software ecosystem support and limited programmability have significantly constrained the widespread deployment of such hardware accelerators. CPUs, owing to their versatility and widespread applicability, remain strong candidates for DNN acceleration. Several CPU vendors have proposed matrix extensions, but none has publicly disclosed the detailed microarchitecture, and researchers also lack simulation tools for evaluating architectures based on matrix extensions during early-stage design. To address these challenges, we propose RVME, an efficient matrix engine based on a matrix extension of RISC-V, designed as a CPU coprocessor, along with an open-source and configurable simulator built upon gem5. RVME introduces scale-out Outer Product Arrays (OPAs) that achieve bubble-free General Matrix Multiplication (GEMM) execution and superior power efficiency. We also introduce a cache-aware, loop-adaptive mapping framework for RVME that searches for mappings with optimal Energy-Delay Product (EDP). Experimental results show that RVME achieves up to $13.4\times$ speedup and over $21.7\times$ instruction count reduction compared to a RISC-V Vector Extension (RVV)-based design. Additionally, it delivers a peak energy-area efficiency of $1921.4\text{ GOPS/W/mm}^2$, surpassing state-of-the-art DNN accelerators by more than $6\times$.

Index Terms—RISC-V, gem5, matrix extension, DNN

I. INTRODUCTION

Deep Neural Networks (DNNs) are extensively utilized across a wide range of domains such as computer vision and natural language processing. As state-of-the-art models grow from millions to hundreds of billions of parameters, their computational demands scale accordingly, placing unprecedented strain on existing hardware. General Matrix Multiplication (GEMM), as one of the most time-consuming and performance-critical kernels in DNN workloads, has driven the development of various hardware accelerators, such as Domain-Specific Architectures (DSAs) [1]–[3] and GPUs. However, these specialized platforms often suffer from high deployment costs, primarily due to limited programmability and fragmented software ecosystems. Consequently, CPUs continue to serve as a foundational platform for extensive research in DNN acceleration [3]–[8].

This work was supported by AIR Project from Alibaba DAMO Academy and National Key R&D Program of China under Grant No.2023YFB4404100. Corresponding author: Qin Wang.

To meet the growing demand for efficient DNN acceleration, several CPU vendors have introduced Instruction Set Architecture (ISA) extensions specifically designed to accelerate matrix operations. Notable examples include Arm’s Scalable Matrix Extension (SME) [9], Intel’s Advanced Matrix Extensions (AMX) [10], and IBM’s Matrix-Multiply Assist (MMA) [8]. However, none of these vendors have publicly disclosed the microarchitectural details of their matrix extension hardware. RISC-V, an open-source ISA gaining significant traction in both academia and industry, has recently introduced a matrix extension proposal, known as *Attached Matrix Extension (AME)* [6]. Owing to its open instruction set and software ecosystem, RISC-V offers a solid foundation for flexible design exploration and implementation of matrix extension-based hardware.

Systolic array (SA) has long been regarded as an efficient structure for accelerating GEMM [2], due to its high throughput and scalability. However, in the CPU domain, it often suffers from high latency and under-utilization. Recent work [5] proposes a fully pipelined SA to improve performance, at the cost of area and control overhead. To address these limitations, we propose RVME, a decoupled matrix engine as a CPU coprocessor, featuring scale-out *Outer Product Arrays* (OPAs) to alleviate under-utilization with simple control. Furthermore, we present a highly configurable simulator built on gem5 [11] that facilitates efficient design space exploration of RVME across multiple deployment scenarios and application domains.

Mapping strategies play a critical role in optimizing both performance and energy efficiency. Although several prior works have focused on GEMM workloads [7], [12], they primarily target architectures with fixed dataflows and their modeling of data movement is not suitable for architectures that employ caches as the primary on-chip memory. To fill these gaps, we propose a cache-aware, loop-adaptive mapping framework tailored to RVME architecture. In addition, we provide dedicated APIs that allow flexible configuration of matrix sizes and support efficient matrix operations.

To validate the efficiency of the proposed RVME, we evaluate its performance on several representative DNN models using our extended gem5 simulator. Compared with a state-of-the-art vector engine [13] based on RISC-V Vector Extension (RVV), RVME achieves up to a $13.4\times$ speedup and over

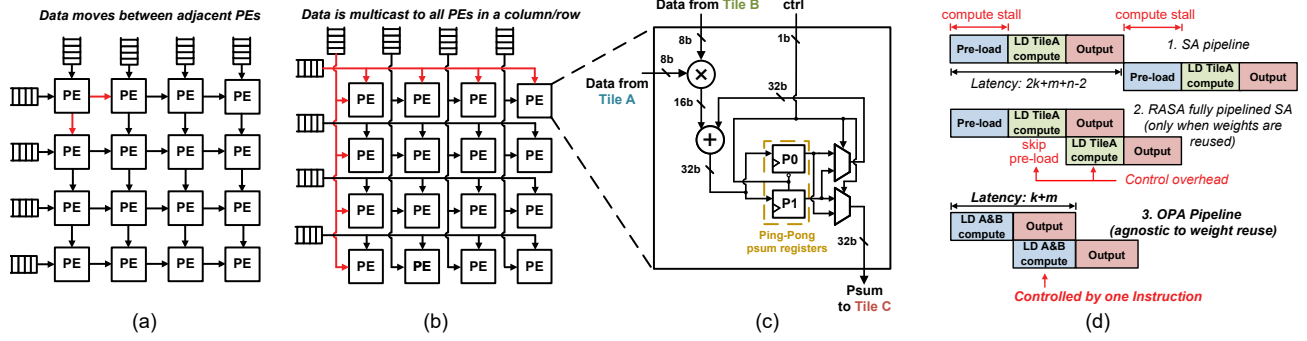


Fig. 1: SA and OPA Architectures. (a)-(b) SA and OPA architectures. (c) PE microarchitecture in OPA. (d) Pipeline diagrams of SA and OPA.

TABLE I: Instructions supported by RVME.

Instruction	Description
<code>mmacc</code>	Matrix multiply-accumulate: $C = C + A \times B$ (A, B: INT8, C: INT32)
<code>mld/mst<a/b/c></code>	Matrix load (mem→reg)/store (reg→mem).
<code>madd/msub/mmul</code> <code>mmax/mmin/mshift</code>	Matrix element-wise operation (INT32).
<code>mcfg<m/n/k></code>	Set matrix size for following operations.
<code>mzero</code>	Set target accumulator register to zero.

a $21.7\times$ reduction in instruction count. Experiments across diverse GEMM workloads show that RVME achieves an improvement of over $8.8\times$ in average area efficiency compared to state-of-the-art accelerators. Ultimately, our design achieves a peak energy-area efficiency of $1921.4 \text{ GOPS/W/mm}^2$ at 1 GHz, surpassing state-of-the-art DNN accelerators by more than $6\times$. Our contributions are summarized as follows:

- We propose RVME, an efficient matrix engine adopting bubble-free OPAs based on a matrix extension of RISC-V, along with a configurable open-source¹ simulator built on gem5.
- We present a cache-aware, loop-adaptive mapping framework that automatically finds optimal mappings for diverse GEMM workloads, exposed through dedicated APIs for efficient hardware execution.
- Our evaluation on representative DNN workloads shows that RVME achieves superior speedup and instruction reduction compared to a RVV-based vector engine. Furthermore, RTL implementation for RVME validates its significant improvements in energy-area efficiency over state-of-the-art accelerators.

II. INSIGHT

A. RISC-V needs Matrix Extension

Although RVV [14] is primarily designed for High-Performance Computing workloads, it falls short in effectively supporting the highly structured and data-intensive computation patterns characteristic of DNNs. This mismatch has motivated the RISC-V community to define a dedicated matrix

extension tailored for efficient DNN acceleration. Similar to other matrix extensions, AME introduces independent matrix registers which extend the data operation granularity from scalar/vector to matrix. Inspired by [15], we introduce a matrix extension of RISC-V, which conforms to the design principles outlined by AME, with its instructions listed in Table I.

We introduce eight architectural matrix registers in RVME, denoted as m_0-m_7 . Among them, m_0-m_3 are Tile Registers (TR) which store the input matrices A and B of GEMM operation, while m_4-m_7 are Accumulator Registers (AR) which store the output matrix C. A single `mmacc` instruction reads two matrices from the TR as inputs and performs the accumulation with the values previously stored in the AR. The matrix size configured by `mcfg` applies to both memory and arithmetic operations. Matrices A and B are defined as INT8, and matrix C is defined as INT32. Sufficient encoding space is reserved for additional data types in the future.

B. Outer Product Array is better for CPU

SAs are recognized as an efficient spatial architecture for GEMM acceleration [2]. However, integrating SA into CPU coprocessor introduces substantial challenges:

High latency. Fig. 1(a) illustrates the topology of a typical SA. In a SA, each processing element (PE) receives data from both the horizontal (X) and vertical (Y) directions and performs a multiply-accumulate (MAC) operation. It then forwards the data to its neighboring PEs for computation in the next cycle. However, the inter-PE data transfer pattern not only leads to high latency, but also introduces unnecessary power consumption.

Under-utilization. Considering a typical SA with Weight Stationary (WS) dataflow, the execution generally involves three phases, as illustrated in Fig. 1(d): (1) *Pre-load Tile B*: The matrix tile B is pre-loaded into the SA. (Here we assume Tile B represents DNN weights.) (2) *Load Tile A and Compute*: Data from Tile A is streamed into the SA, and this input phase continues until the first row of the SA has received all its required data elements from Tile A. (3) *Output*: As Tile A continues streaming into the array, psums accumulate and propagate downward along the Y direction until the computation completes. In each phase, a subset of PEs

¹Available at <https://github.com/superboy999/RVME>

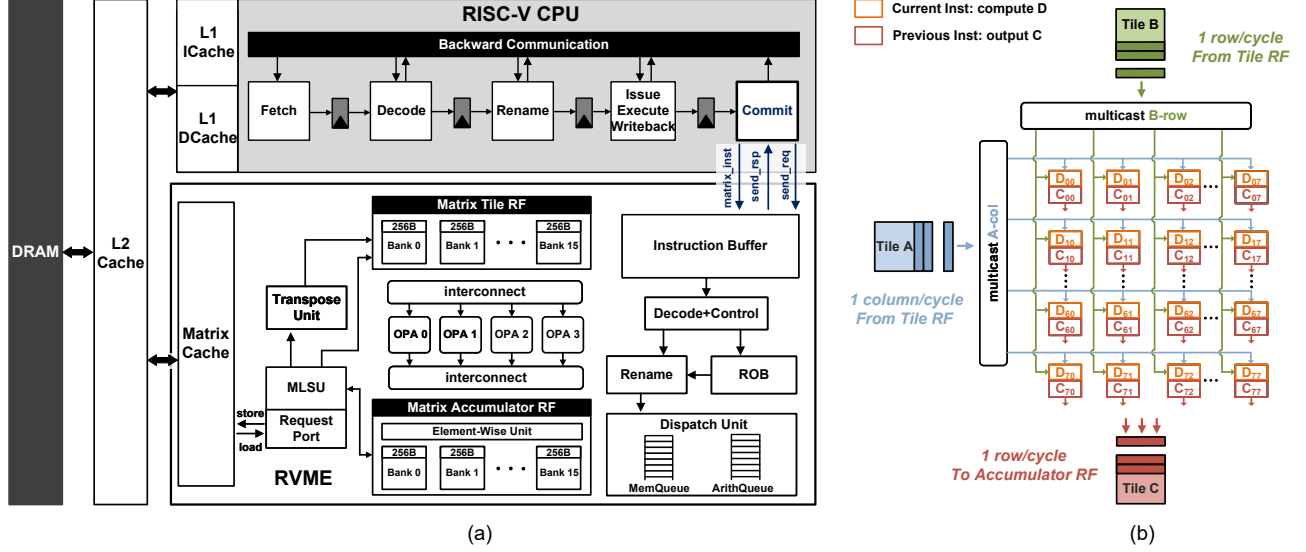


Fig. 2: RVME Architecture Overview (a) and OPA Compute Scheme (b).

become idle, particularly during the pre-load stage, resulting in overall under-utilization of the SA. The SA with Output Stationary (OS) dataflow exhibits a similar trend. This under-utilization is further worsened when weights cannot be reused between two consecutive matrix MAC operations.

While RASA [5] introduces a fully pipelined SA, this design incurs substantial area and instruction scheduling overhead due to its structural and control complexity.

Taking all the above into consideration, we introduce an OPA architecture, as illustrated in Fig. 1(b). During each cycle, a column from matrix A and a row from matrix B are fed into the OPA, while each data element is multicast to all PEs along its corresponding row or column. Each PE performs MAC operations, with the resulting partial sums (psums) being accumulated locally until the entire K dimension is traversed. This multicast-based dataflow eliminates inter-PE communication compared to SA, and removes the need for input registers in both the X and Y directions within each PE.

Moreover, OPA architecture also removes the need for an explicit pre-load phase and sustains high PE utilization through simple control. As shown in Fig. 1(c), we propose a novel PE design for the OPA that supports fully pipelined execution. This PE incorporates a pair of ping-pong psum registers controlled by a 1-bit flag. Once a MAC operation is completed, the OPA can immediately accept the next instruction without waiting for the previous output. The newly generated psum values are stored in an alternate register, while the previous register simultaneously outputs the psum, removing OPA bubbles caused by waiting for output. The corresponding pipeline diagram is illustrated in Fig. 1(d).

We implement 8×8 configuration for both SA and OPA, operating at 1 GHz. Although the OPA architecture introduces higher fan-out in the data paths, the critical-path delay in our implementation increases by only 3.5%, with an area overhead of 4.1% due to the additional psum register. Nevertheless, OPA

achieves a 33% reduction in power consumption and reduces the latency of matrix MAC from $2k+m+n-2$ [16] to $k+m$ (k for MAC computation, m for result output), where m , n , and k denote the tile sizes mapped onto the OPA and SA. Notably, the OPA achieves bubble-free execution provided that the input matrix dimensions satisfy $k \geq m$. This constraint is inherently met by the dimension of the matrix registers, and the tiling strategy strives to maximize their utilization. Meanwhile, as the size of the OPA increases, the power consumption associated with data multicasting and the increase in critical-path delay become non-negligible. Therefore, rather than adopting a scale-up approach with one monolithic OPA, RVME follows a scale-out strategy that deploys four OPAs in parallel.

III. RVME DESIGN METHODOLOGY

A. RVME Architecture Overview

Fig. 2(a) presents an overview of RVME architecture. As a decoupled coprocessor, RVME has a dedicated matrix cache, which shares the same L2 cache with host CPU. It integrates an independent decode unit and a Matrix Load/Store Unit optimized for efficient matrix access. RVME integrates a multi-bank matrix register file, four OPAs each with their PE array size set to 8×8 , and a mapping exploration framework dedicated for GEMM workload. These components work jointly to facilitate efficient GEMM execution. Additionally, RVME incorporates register renaming and a Reorder Buffer (ROB) to support out-of-order execution and register-level prefetching, effectively mitigating pipeline stalls.

Instruction Offload. As shown in Fig. 2(a), we choose the Out-of-Order (OoO) CPU in gem5 as the host processor, to ensure that RVME is evaluated without being limited by the instruction offload throughput. The OoO CPU in gem5 consists of five pipeline stages: Fetch, Decode, Rename, Issue/Execute/Writeback (IEW), and Commit. To minimize modifications to the host CPU pipeline, matrix instructions

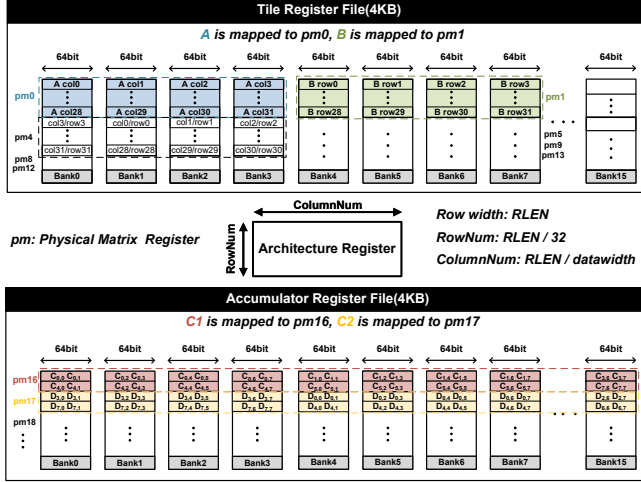


Fig. 3: Matrix Register File Organization and Data Mapping.

will be treated as "Executed" and "CanCommit". Furthermore, considering the significant penalty incurred by pipeline flushes within RVME, matrix instructions will not be offloaded to RVME until they reach the head of ROB in the host CPU. Additionally, we introduce a dedicated instruction buffer which temporarily holds matrix instructions, allowing them to be promptly offloaded and decoupled from the CPU.

Out-of-Order support. A limited number of registers may cause OPAs to become idle when the required data is unavailable. To mitigate this, we adopt 32 physical registers and a renaming unit to pre-load data into registers for subsequent compute instructions. Given that RVME supports both in-order and out-of-order execution modes, a ROB is implemented to enable out-of-order execution while ensuring in-order instruction commit. In this work, we adopt a conservative strategy for releasing physical registers. Specifically, if two instructions sequentially target the same logical destination register, the physical register allocated to the earlier one will not be released until the later instruction commits.

Matrix Register File. A matrix register is designed with each row having a width of $RLEN$ bits and $RLEN/32$ rows, ensuring that the psm matrix fits entirely within the register. Matrix Register File (MRF) is implemented using a multi-bank architecture, with each bank equipped with one read port and one write port. Fig. 3 illustrates the data mapping scheme of the matrix register. Each TR is interleaved across four SRAM banks to accommodate 32-byte data inputs from both the Matrix Load/Store Unit (MLSU) and the transpose unit. Since each element in the AR is 32-bit, each row is interleaved across four SRAM banks. Unlike the TR, the AR adopts a different mapping strategy in which the entire register is distributed across 16 banks to reduce bank conflicts, as it may receive simultaneous write outputs from four OPAs.

Matrix Load/Store Unit. MLSU manages data transfers between RVME and the memory hierarchy. To maintain sustained data throughput, we optimize data layout at the software level by allocating data with cache line alignment and applying

TABLE II: Transfer Factors in Data Movement Analysis

Perm.	A'_n	B'_n	C'_n	P'_n (a)
MNK	$\frac{n_{tn}}{n_{tn-1}}$	$\frac{m_{tn}}{m_{tn-1}}$	1	$2\left(\frac{k_{tn}}{k_{tn-1}}\right) - 1$
NMK	$\frac{n_{tn}}{n_{tn-1}}$	$\frac{m_{tn}}{m_{tn-1}}$	1	$2\left(\frac{k_{tn}}{k_{tn-1}}\right) - 1$
MKN	1	$\frac{m_{tn}}{m_{tn-1}}$	$2\left(\frac{k_{tn}}{k_{tn-1}}\right) - 1$	$2\left(\frac{k_{tn}}{k_{tn-1}}\right) - 1$
KMN	1	$\frac{m_{tn}}{m_{tn-1}}$	$2\left(\frac{k_{tn}}{k_{tn-1}}\right) - 1$	$2\left(\frac{k_{tn}}{k_{tn-1}}\right) - 1$
NKM	$\frac{n_{tn}}{n_{tn-1}}$	1	$2\left(\frac{k_{tn}}{k_{tn-1}}\right) - 1$	$2\left(\frac{k_{tn}}{k_{tn-1}}\right) - 1$
KNM	$\frac{n_{tn}}{n_{tn-1}}$	1	$2\left(\frac{k_{tn}}{k_{tn-1}}\right) - 1$	$2\left(\frac{k_{tn}}{k_{tn-1}}\right) - 1$

(a) $P'_n = 1$ when $m_{tn} = m_{tn-1}$ and $n_{tn} = n_{tn-1}$.

data reordering for matrices A and B. To improve memory access efficiency, we introduce two 64-byte buffers in MLSU to match the cache line size (64-byte) with memory access width (32-byte) during both load and store operations.

Control and Compute Scheme. To achieve high utilization, RVME employs an efficient execution pipeline. The instruction buffer continuously accepts matrix instructions while it is not full. When free physical registers exist and the ROB is not full, the rename unit automatically maps the destination registers of incoming instructions to available physical registers. Subsequently, memory and arithmetic instructions are dispatched into two independent queues.

The MLSU sends matrix A to the transpose unit, converting it into column-major format, while matrices B and C are directly loaded to the MRF. Once MAC operation of a previous instruction completes, the OPA can be assigned a new MAC instruction. The corresponding SRAM banks for the input operands will be accessed sequentially, as illustrated in Fig. 2(b). Each OPA outputs one row of PE results per cycle to the Element-Wise Unit (EWU), which performs accumulation and writes the results to the destination AR. Notably, OPAs work in pipeline-manner, the latency of this entire output process can be effectively hidden by the MAC operations of a subsequent instruction. Meanwhile, four OPAs can execute MAC operation in parallel once their input operands are ready.

B. Cache-aware, Loop-adaptive Mapping Framework

Some prior works have explored techniques to reduce data movement across the memory levels in GEMM operations [7], [12]. However, these approaches either employ data reuse models that are unsuitable for architectures where caches serve as the primary on-chip memory, or they fail to provide flexible mapping for architectures with configurable dataflow. Inspired by [17], we propose an efficient cache-aware mapping exploration framework with a loop-adaptive search step, specifically tailored for GEMM workloads on RVME, designed to search for the Energy-Delay Product (EDP) optimal tiling and its corresponding dataflow.

Data movement analysis. GEMM can be described as a three-dimensional nested loop over M , N , and K . It can be regarded as a simplified form of convolution that reduces the number of independent dimensions from six (excluding batch size) to three, thereby significantly shrinking the mapping space and simplifies the analysis of data reuse. Once the loop order and tile sizes are determined, the data movement

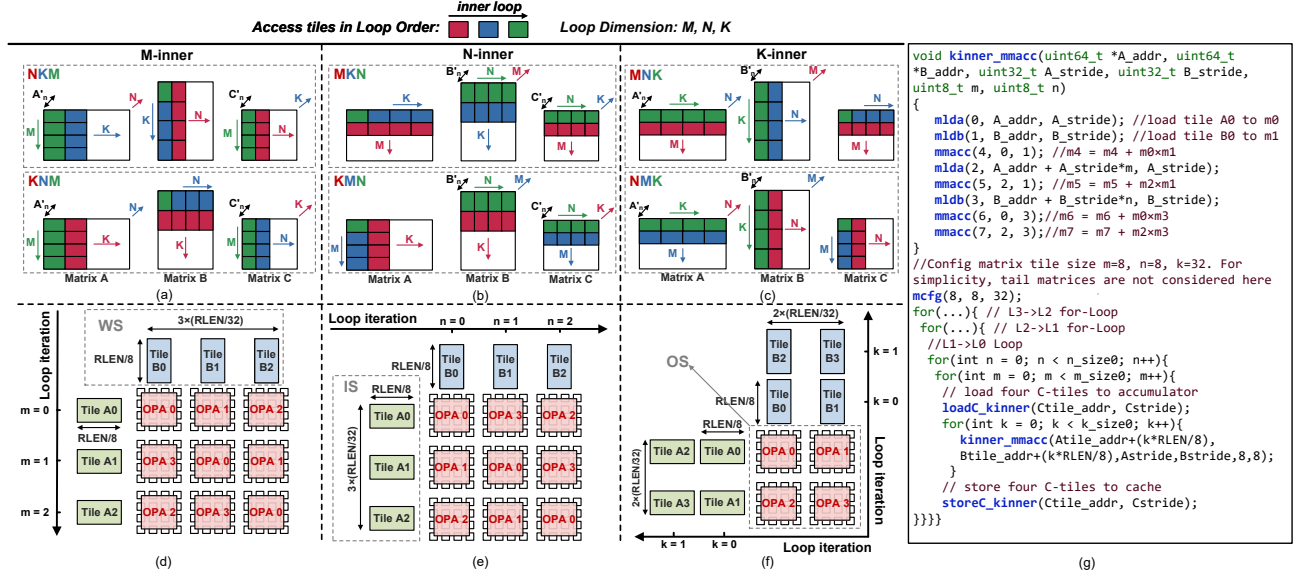


Fig. 4: Different data movement of loop permutations and scheduling strategies for RVME under three dataflows. (a)–(c) Data movement between L_n and L_{n-1} under three dataflows, disregarding cross-level effects. (d)–(f) Three scheduling strategies for scale-out OPAs under corresponding dataflow. (g) Example Code for OS dataflow in RVME.

between two memory levels can be formally computed without tracking delta tiles across iterations [17].

With three independent loop dimensions, the six possible loop permutations are shown in Fig. 4(a)–(c). These permutations can be categorized into three classes based on their data reuse patterns across adjacent memory levels: *K-inner*, *N-inner*, and *M-inner*. Permutations with *K* as the innermost loop are defined as *K-inner*, which correspond to the *Output Stationary (OS)* dataflow, since the psums of matrix *C* remain in the next memory level until the *K*-loop fully iterated. Similarly, *N-inner* and *M-inner* correspond to *Input Stationary (IS)* and *WS* dataflows, respectively.

Memory hierarchy of RVME comprises four levels: L_0 (MRF), L_1 (Matrix Cache), L_2 (L2 Cache), and L_3 (DRAM), as shown in Fig. 5(a). Consider a GEMM workload $C = A \times B$, where the matrices A, B and C are of sizes $M \times K$, $K \times N$ and $M \times N$. For clarity, we define the tile sizes at memory level L_n as $m_{tn} \times k_{tn}$, $k_{tn} \times n_{tn}$, and $m_{tn} \times n_{tn}$, representing the dimensions of a tile of matrices A, B, and C, respectively. Considering the temporal locality of data in caches², we conservatively assume that, at memory level L_n , only one tile from each of matrices A, B, and C resides in that level during each loop iteration. Fig. 4(a)–(c) illustrates the data movement between memory levels L_n and L_{n-1} under different loop permutations, without considering the influence of data movement across memory levels. Evidently, permutations that share the same dataflow lead to identical data movement. Further, when considering data movement across memory levels, for the transfer of matrices A and B between L_n and L_{n-1} , the transfer factors A'_n and B'_n , which represent the number of times the entirety of matrix A and

matrix B are transferred, have to be propagated downward into the computation of data movement between L_{n-1} and L_{n-2} and subsequent levels. For *M-inner* and *N-inner* permutations, the transfer factor C'_n for matrix *C* between L_n and L_{n-1} is $2(k_{tn}/k_{tn-1}) - 1$, which because a tile of matrix *C* must first be loaded into the inner memory level and then stored back after accumulation during each innermost iteration. However, for the *K-inner* permutation, where psums remain in L_{n-1} until the *K* dimension is fully iterated, the corresponding transfer factor C'_n is 1. However, due to our conservative assumption, a factor of $2(k_{tn}/k_{tn-1}) - 1$, which we denote as P_n , is still propagated downward into the data movement computation between L_{n-1} and L_{n-2} . The transfer factors for all six loop permutations are summarized in Table II. Taking all the above analysis into account, the total data movement in bytes between memory levels L_n and L_{n-1} can be expressed as:

$$DM_n = \left(\prod_{i=n}^{L_h} A'_i \right) MK + \left(\prod_{i=n}^{L_h} B'_i \right) KN + \left(\prod_{i=n+1}^{L_h} P_i \right) \left(\prod_{i=n}^{L_h} C'_i \right) \cdot 4MN \quad (1)$$

where L_h represents the outermost level of the memory hierarchy, and $4MN$ denotes the total size in bytes of matrix *C*, where each element is 4-byte.

Loop-adaptive Search and Mapping strategy. An intuitive strategy to maximize the utilization of MRF and OPAs while avoiding unnecessary exploration in the mapping space is to align the tiling search steps with the matrix register size. However, we introduce distinct scheduling strategies tailored for three dataflows in RVME to fully exploit MRF and scale-

²We assume all caches adopt the Least Recently Used replacement policy.

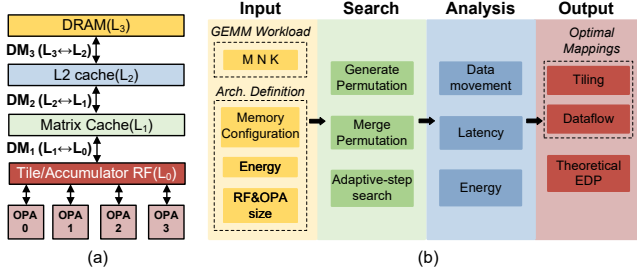


Fig. 5: RVME memory hierarchy and framework workflow.

out OPAs. Consequently, our framework’s adaptive search step is configured to the equivalent tile sizes corresponding to each dataflow in RVME.

Since the dataflow in RVME is determined by the data movement pattern between L_1 and L_0 . We partition the mapping space into three sub-spaces based on the loop permutation between L_1 and L_0 , and for each sub-space, we employ its corresponding equivalent tile size as the search step, as illustrated in Fig. 4(d)–(f). For the K -inner permutation between L_1 and L_0 , each iteration loads two tiles from matrix A (e.g., A_0 and A_1) and two tiles from matrix B (e.g., B_0 and B_1), which perform MAC operations pairwise, with the computations mapped in parallel onto four OPAs. The resulting four tiles of matrix C are stationary in AR until the K -loop completes its full iteration. In this case, the tile search steps are configured as $m_{\text{step}} = n_{\text{step}} = RLEN/16$, and $k_{\text{step}} = RLEN/8$, also represents the equivalent tile size per iteration. For the M -inner, three tiles of matrix B are pre-loaded and stationary in TRs throughout the entire M -loop iteration. In each iteration, one tile of matrix A is loaded and used to perform three matrix MAC, while the rest OPA is allocated for computation in the subsequent iteration. The corresponding tile search steps are configured as $m_{\text{step}} = RLEN/32$, $n_{\text{step}} = 3 \cdot RLEN/32$, and $k_{\text{step}} = RLEN/8$. For the N -inner permutation, three tiles from matrix A are retained in matrix registers, while one tile from matrix B is loaded in each iteration.

We also provide dedicated APIs to execute matrix operations with configurable sizes and dataflows. An code example for mapping a GEMM workload with OS dataflow onto RVME is shown in Fig. 4(g).

Framework Workflow. The workflow of the proposed framework is depicted in Fig. 5(b). Initially, the framework takes the GEMM workload and architecture definition as input and proceeds to enumerate all feasible loop permutation throughout the memory hierarchy. In the subsequent search phase, permutations that result in identical data movement patterns are merged. To establish a valid mapping space, we consider only tile sizes that fit within the capacity of the inner memory level when exploring valid tilings between two adjacent memory levels. By optimizing the data layout in software, each memory request is guaranteed to access data within a single cache line, avoiding cache bank conflicts and simplifying the calculation of cache latency and energy. Energy consumption and latency for whole system are estimated following the methodology outlined in [17]. Ultimately, the

TABLE III: Architecture Configuration of RVME

Host CPU	Clock Frequency: 2 GHz	
	64-bit RISC-V OoO CPU	
RVME	Clock Frequency: 1 GHz	
	MRF: 8KB, RLEN = 256-bit	
	Physical Registers: 32 (16 Tile, 16 Accumulator)	
	OPA size: $1 \times 8 \times 8$	OPA size: $4 \times 8 \times 8$
	Vector Comparison Setting	Default Setting
	Instruction Buffer Entries: 32	
	ArithQueue / MemQueue Entries: 16	
Memory System	Matrix Cache: 32KB, Cache Line size: 64B	
	L1 I-Cache / L1 D-Cache: 32KB, Cache Line size: 64B	
	L2 Cache: 256KB, Cache Line size: 64B	
	DDR4 Memory: 2 GB	

framework delivers optimal mappings, which include tiling and dataflow, achieving a theoretically optimal EDP.

IV. EVALUATION

We implement RVME on our extended gem5 simulator to evaluate its performance, and the corresponding system configuration is summarized in Table III. For these evaluations, GEMM workloads targeting RVME are optimized using our proposed mapping framework. To quantitatively demonstrate the advantage of the proposed matrix extension over RVV, we evaluate RVME and an RVV-based vector engine using four representative DNNs. Additionally, we implement RVME in SystemVerilog to demonstrate its efficiency against other representative accelerators.

A. Compared with RVV

We extend the RISC-V GNU toolchain to support our custom matrix extension listed in Table I. And we implement four representative DNN models [18]–[21] using matrix extension and RVV, focusing evaluation on layers that can be transformed into GEMM operations. For convolution operations in ResNet-50 and MobileNetV2, we use the im2col algorithm to convert convolution layers into GEMM operations with the batch size of 1. Additionally, we use a state-of-the-art RVV-based vector engine [13] as the baseline. For a fair comparison, both VLEN and RLEN are set to 256 bits. Under the same memory configuration, RVME is configured with one OPA, while the vector engine is configured with eight lanes. This ensures that the datapath width is the same for both, each consuming 16 input data per cycle.

Fig. 6 demonstrates a reduction of over $21.7\times$ in the total instruction count achieved by the matrix extension. This is primarily because a single `mmacc` instruction performs $64\times$ more operations than a `vmacc` instruction, significantly reducing the number of arithmetic instructions count. Configuration and memory instructions are also reduced, as each matrix instruction processes larger data blocks, thereby reducing the frequency of control and memory access operations. RVME achieves speedups ranging from $7.9\times$ to $9.4\times$ for Transformer-base and BERT-base models, while for ResNet-50, the speedup is between $8.5\times$ and $13.4\times$. In MobileNetV2, due to the characteristics of depthwise separable convolutions, the GEMM

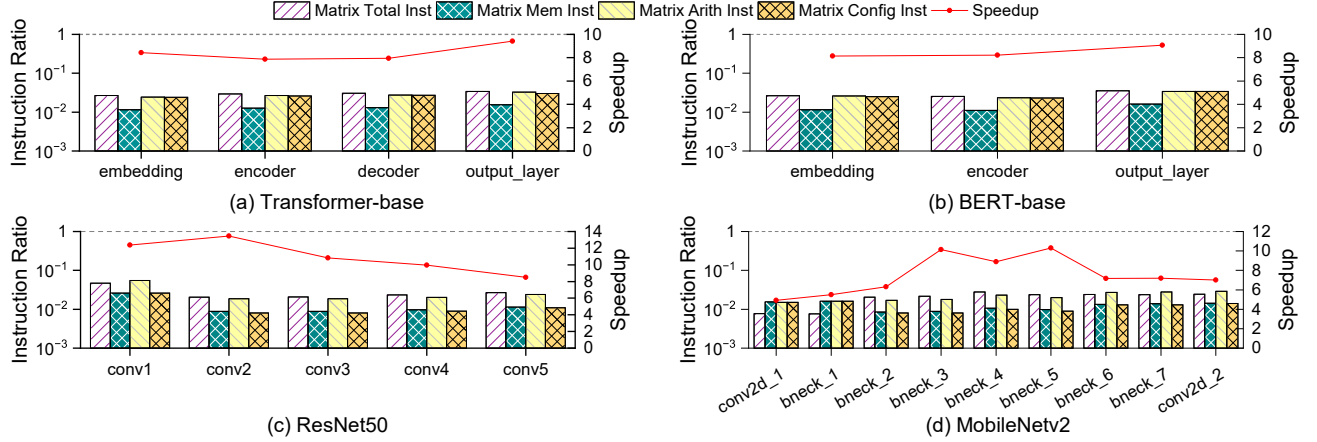


Fig. 6: Instruction ratio (Matrix Extension/RVV) across four DNN models with RVV normalized to 1 (left y-axis). The speedup of RVME relative to the vector engine using RVV is shown (right y-axis) under identical memory configuration, with vector length (VLEN) and matrix register length (RLEN) set to 256-bit.

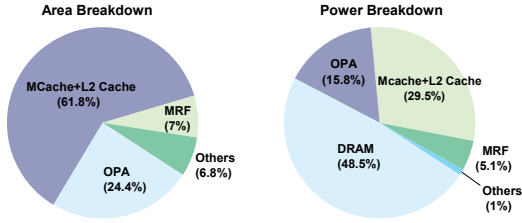


Fig. 7: Area and power breakdown of RVME.

sizes are often too small to fully utilize MRF and OPA. This is particularly evident from the `conv2d_1` layer to the `bottleneck_2` layer, where the observed speedup ranges from $4.9\times$ to $6.3\times$.

B. Area and Power Evaluation

We synthesize RVME using Synopsys Design Compiler with a 28 nm process technology targeting a clock frequency of 1 GHz. To evaluate the power consumption of RVME under real GEMM workloads, we use Synopsys PrimeTime PX and extend McPAT [22]. We obtained memory component parameters, including energy and area, from CACTI [23]. Fig. 7 shows area and power breakdown of RVME.

RVME occupies a total area of 0.63 mm^2 . The matrix cache and shared L2 cache, which together form the main on-chip memory of RVME, account for 61.8% of this. The MRF, implemented with 32 matrix registers, occupies 7%, while the OPAs contribute another 24.4% to the total area. For detailed power analysis, we use McPAT with input trace generated by `gem5`, which is extended with the detailed energy consumption of each component and the overall leakage power of RVME, both derived from PrimeTime PX. Under real GEMM workload, RVME achieves an average power consumption of 0.377 W. The power breakdown reveals that DRAM accesses constitute 48.5% of this total, a significant portion attributable to the constrained on-chip memory capacity. The power consumption of the matrix cache and shared L2 cache accesses is 29.5% of the total, while the OPAs are responsible for 15.8%.

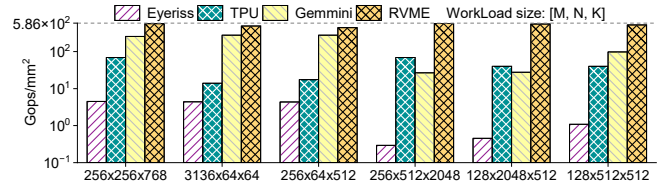


Fig. 8: Comparison of area efficiency across workloads.

C. Compared with Other Accelerators

We select several representative GEMM workloads from the aforementioned DNNs, covering various matrix sizes and shapes. Fig. 8 compares the area efficiency of RVME against several state-of-the-art accelerators, whose performance is modeled using SCALE-SIM [16]. To ensure a fair comparison, we adopt a same DRAM bandwidth setting as RVME for both Gemini [3] (OS) and Eyerriss [1], as neither of them explicitly specifies DRAM bandwidth. RVME achieves a peak area efficiency of 586 GOPS/mm^2 and an average of 527.9 GOPS/mm^2 , as shown in Fig. 8. Compared to Eyerriss, RVME demonstrates over $93\times$ higher area efficiency. This improvement can be attributed to both technology scaling and the poor data reuse of Eyerriss when running GEMM workloads with its row-stationary (RS) dataflow. Additionally, due to flexible dataflow and bubble-free OPAs, RVME achieves an average of $8.8\times$ higher area efficiency than Gemini. In these workloads, the large SA of TPUv1 [2] cannot always be efficiently utilized, leading to low PE utilization and allowing RVME to achieve $17.4\times$ higher average area efficiency.

Additionally, we compare RVME with several state-of-the-art accelerators, whose specifications are summarized in Table IV. In this work, we adopt the metric GOPS/W/mm^2 as a key metric for CPU coprocessor scenarios. This metric is particularly important in constrained hardware environments, as it enables a holistic evaluation of both area and energy efficiency. Experiments show that RVME achieves up to $1921.4 \text{ GOPS/W/mm}^2$ on real GEMM workloads, surpassing other accelerators by over $6\times$.

TABLE IV: Comparison of Specification and Key Efficiency Metrics with State-of-the-Art Accelerators

	Eyeriss [1]	TPUv1 [2]	Gemmini [3]	MECLA [24]	MX [7]	MACO [4]	This Work
Technology(nm)	65	28	22	28	12	12	28
Frequency(MHz)	200	700	1000	1000	910 (64-Core Cluster)	2500	1000
Area(mm ²)	12.25	331	1.03	22.02	-	1.58	0.63
Support Precision	16b Fixed-Point	INT8	INT8	INT8	FP32	FP16,32,64	INT8
Peak Performance(GOPS)	84	91750	512	14008	512	320(FP16)	512
Peak Area Efficiency (GOPS/mm ²)	6.9	245.6	497	636	-	202.4(FP16)	586 ^(a)
Peak Energy Efficiency (GOPS/W)	241.7	2293.8	-	7008	307	213.2(FP16)	1202.1 ^(a)
Peak Energy-Area Efficiency (GOPS/W/mm ²)	20	6.9	-	318.2	-	135(FP16)	1921.4^(a)

(a) Results are obtained from real workloads.

V. CONCLUSION

In this paper, we propose RVME, an efficient coprocessor architecture based on the matrix extension. RVME employs bubble-free OPAs to minimize compute stalls and features a cache-aware, loop-adaptive mapping framework for optimizing GEMM workload mapping. Compared with a RVV-based design, RVME significantly reduces instruction count by 21.7 $\times$, achieving speedups of up to 13.4 $\times$. Furthermore, experiments show that RVME demonstrates a peak energy-area efficiency of 1921.4GOPS/W/mm², surpassing existing state-of-the-art accelerators by over 6 $\times$.

REFERENCES

- [1] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017.
- [2] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proceedings of the 44th annual international symposium on computer architecture*, 2017, pp. 1–12.
- [3] H. Genc, S. Kim, A. Amid, A. Haj-Ali, V. Iyer, P. Prakash, J. Zhao, D. Grubb, H. Liew, H. Mao, A. Ou, C. Schmidt, S. Steffl, J. Wright, I. Stoica, J. Ragan-Kelley, K. Asanovic, B. Nikolic, and Y. S. Shao, "Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*, 2021, pp. 769–774.
- [4] B. Sui, J. Shen, C. Sun, J. Wang, Z. Zheng, and W. Guo, "Maco: Exploring gemm acceleration on a loosely-coupled multi-core processor," in *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2024, pp. 1–6.
- [5] G. Jeong, E. Qin, A. Samajdar, C. J. Hughes, S. Subramoney, H. Kim, and T. Krishna, "Rasa: Efficient register-aware systolic array matrix engine for cpu," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 253–258.
- [6] RISC-V, "attached-matrix-extension," 2023. [Online]. Available: <https://github.com/riscv-admin/attached-matrix-extension>
- [7] M. Perotti, Y. Zhang, M. Cavalcante, E. Mustafa, and L. Benini, "Mx: Enhancing risc-v's vector isa for ultra-low overhead, energy-efficient matrix multiplication," in *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2024, pp. 1–6.
- [8] J. P. de Carvalho, J. E. Moreira, and J. N. Amaral, "Compiling for the ibm matrix engine for enterprise workloads," *IEEE Micro*, vol. 42, no. 5, pp. 34–40, 2022.
- [9] Arm, "Introducing the scalable matrix extension for the armv9-a architecture," 2021. [Online]. Available: <https://community.arm.com/developer/ip-products/processors/b/processors-ip-blog/posts/scalable-matrix-extension-armv9-a-architecture>
- [10] Intel, "Accelerate artificial intelligence (ai) workloads with intel advanced matrix extensions (intel amx)," Intel Corporation, Tech. Rep., 2022.
- [11] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Sardashti, R. Sen, K. Sewell, M. Shoaib, N. Vaish, M. D. Hill, and D. A. Wood, "The gem5 simulator," *SIGARCH Comput. Archit. News*, vol. 39, no. 2, p. 1–7, aug 2011.
- [12] M. Ronzani and C. Silvano, "Factorflow: Mapping gemms on spatial architectures through adaptive programming and greedy optimization," in *Proceedings of the 30th Asia and South Pacific Design Automation Conference*, ser. ASPDAC '25, 2025, p. 706–712.
- [13] C. Ramírez, C. A. Hernández, O. Palomar, O. Unsal, M. A. Ramírez, and A. Cristal, "A risc-v simulator and benchmark suite for designing and evaluating vector architectures," *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 17, no. 4, pp. 1–30, 2020.
- [14] RISC-V, "Working draft of the proposed risc-v v vector extension," 2020. [Online]. Available: <https://github.com/riscv/riscv-v-spec>
- [15] XUANTIE-RV, "riscv-matrix-extension-spec," 2022. [Online]. Available: <https://github.com/XUANTIE-RV/riscv-matrix-extension-spec>
- [16] A. Samajdar, J. M. Joseph, Y. Zhu, P. Whatmough, M. Mattina, and T. Krishna, "A systematic methodology for characterizing scalability of dnn accelerators using scale-sim," in *2020 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2020, pp. 58–68.
- [17] A. Parashar, P. Raina, Y. S. Shao, Y.-H. Chen, V. A. Ying, A. Mukkara, R. Venkatesan, B. Khailany, S. W. Keckler, and J. Emer, "Timeloop: A systematic approach to dnn accelerator evaluation," in *2019 IEEE international symposium on performance analysis of systems and software (ISPASS)*. IEEE, 2019, pp. 304–315.
- [18] A. Vaswani, "Attention is all you need," *Advances in Neural Information Processing Systems*, 2017.
- [19] J. Devlin, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [20] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [21] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 4510–4520.
- [22] S. Li, J. H. Ahn, R. D. Strong, J. B. Brockman, D. M. Tullsen, and N. P. Jouppi, "Mcpat: An integrated power, area, and timing modeling framework for multicore and manycore architectures," in *Proceedings of the 42nd annual ieee/acm international symposium on microarchitecture*, 2009, pp. 469–480.
- [23] N. Muralimanohar, R. Balasubramanian, N. P. Jouppi *et al.*, "Cacti 6.0: A tool to model large caches," *HP laboratories*, vol. 27, p. 28, 2009.
- [24] Y. Qin, Y. Wang, Z. Zhao, X. Zhao, Y. Zhou, S. Wei, Y. Hu, and S. Yin, "Mecla: Memory-compute-efficient llm accelerator with scaling sub-matrix partition," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024, pp. 1032–1047.